

Department of Computer Applications

Master of Computer Applications (MCA)

Final Year Project — Semester III

Literature Review and Research Understanding Report

ZeroTrustCloud / TrustMesh

A Review of 10 Research Papers on Identity-Centric
Zero-Trust Architecture and Supply Chain Security

Project Research Context

Modern enterprise applications are no longer confined within a physical data center. With cloud-native deployments on platforms like AWS EKS and Azure AKS, workloads run as short-lived containers, spinning up and down across shared compute. The old model of security where you trusted anything inside the corporate network simply does not hold anymore. Once an attacker gets through the outer boundary, they can move freely across an unsegmented cloud environment.

Zero Trust Architecture (ZTA) is the response to this problem. Its core principle is straightforward: 'never trust, always verify.' Instead of trusting based on network location, every access request from a user, a microservice, or a CI/CD pipeline must present a verified identity and must be authorized based on policy before it is allowed. Beyond that, the security problem does not stop at runtime. The software supply chain is also under threat. CI/CD pipelines, container registries, and third-party open-source packages are all potential entry points for attackers and if malicious code enters the build pipeline, runtime controls mean nothing.

Our project, ZeroTrustCloud, aims to design and implement a practical zero-trust framework called TrustMesh for Kubernetes-based cloud environments. The main areas we are focusing on are: workload identity using SPIFFE/SPIRE and OIDC, policy-based authorization using Open Policy Agent (OPA), service mesh security (mTLS) using Istio, and supply chain protection through CI/CD pipeline hardening and artifact signing.

To properly scope and ground our work, we reviewed 10 research papers that are directly relevant to these areas. The goal of this review is not just to summarize what the papers say, but to understand what the authors actually did, what worked, what did not work, and what gaps are still open for us to address in TrustMesh.

Paper 1 — A Multi-Cloud Framework for Zero-Trust Workload Authentication

Paper Details

- **Title:** A Multi-Cloud Framework for Zero-Trust Workload Authentication
- **Authors:** Saurabh Deochake, Ryan Murphy, and Jeremiah Gearheart (SentinelOne Inc.)
- **Year / Publisher:** 2024 | SentinelOne Inc. Technical Research Publications
- **Research Area:** Multi-Cloud Workload Identity Federation, OIDC, Ephemeral Token Authentication

Problem Addressed

The central problem this paper deals with is something very common in cloud environments: workloads authenticating to other cloud services using static, long-lived IAM keys or service account private keys. These keys get stored on disk or inside Kubernetes secrets, and if they are leaked or stolen, an attacker can use them for a long time without anyone knowing. In a large deployment with over 100 Kubernetes clusters, managing and rotating these keys is also practically impossible to do reliably.

Approach / Methodology

The authors built a Workload Identity Federation (WIF) framework using OpenID Connect (OIDC) and Kubernetes Bound Service Account Tokens. The idea is that instead of storing a private key, each pod gets a short-lived JWT from the Kubernetes API server and exchanges it with the cloud provider's Security Token Service (STS) for temporary credentials that expire in under 60 minutes. They tested this at real enterprise scale — across more than 100 clusters in 10+ cloud regions.

What We Understood

The biggest takeaway from this paper is that static credentials are inherently a Zero Trust violation, because they represent pre-granted, persistent trust rather than continuously verified identity. The use of audience (aud) scoping in OIDC tokens is particularly important — it prevents one cloud's token from being replayed against a different cloud service (the 'confused deputy' attack). Practically, the paper shows that by switching to ephemeral tokens, credential lifetime drops to under an hour and audit complexity goes down significantly. The authors also note that centralizing identity federation means you manage a constant number of trust relationships with cloud IdPs rather than linearly scaling per-key rotations.

Relevance to Our Project

This paper is basically the foundation for TrustMesh's identity layer. It gives us a tested, working model for how AWS EKS and Azure AKS pods can authenticate across cloud boundaries without any stored private keys. We plan to follow the same OIDC federation approach for our workloads.

What We Plan to Apply

- Use Kubernetes Bound Service Account tokens + OIDC federation for secretless workload authentication.
- Strictly validate aud and sub claims in IAM trust policies to prevent cross-service token misuse.
- Keep credential lifetimes short (under 60 minutes) as a baseline.

Limitations / Gap

The paper only addresses authentication — it does not cover what happens after a workload is authenticated. There is no discussion of fine-grained authorization, Policy-as-Code, or how to check whether the build artifact running inside the pod is actually trustworthy. Supply chain integrity is completely out of scope.

Paper 2 — Establishing Workload Identity for Zero Trust CI/CD: From Secrets to SPIFFE-Based Authentication

Paper Details

- **Title:** Establishing Workload Identity for Zero Trust CI/CD: From Secrets to SPIFFE-Based Authentication
- **Author:** Surya Teja Avirneni (Cloud Platform and Security Architect)
- **Year / Publisher:** April 2025 | Independent Cloud Security Architecture Research Report
- **Research Area:** CI/CD Pipeline Security, SPIFFE/SPIRE, Workload Identity, Policy-as-Code

Problem Addressed

This paper looks at a specific and quite serious problem: CI/CD pipelines are treated as trusted administration channels, but they should not be. Pipelines run as highly privileged automation agents that can push code to registries and deploy to Kubernetes, yet they often authenticate using static secrets stored in GitHub repository variables, or shared runner tokens that every job shares. These are the weakest links in the chain — if a supply chain attack compromises a runner, the attacker gets cloud access under legitimate-looking credentials.

Approach / Methodology

The author traces the evolution of CI/CD authentication from stored secrets, to platform-native OIDC (like GitHub Actions OIDC), and then to SPIFFE/SPIRE as an infrastructure-neutral standard. The key idea with SPIFFE is that every workload — including a CI/CD runner — gets a unique, cryptographic, short-lived identity document called an SVID, issued only after the node and workload are attested (verified by runtime selectors like container image digest or service account name). These SVIDs are then paired with OPA or Cedar for actual authorization decisions.

What We Understood

- Pipelines must be treated as untrusted workloads, not trusted admins. The paper makes a very clear argument that any pipeline that can deploy to production is as sensitive as any user account.
- GitHub-native OIDC tokens have a fragility problem: the claim structure is platform-specific and can break when moving across cloud providers. SPIFFE solves this with a standardized URI naming scheme (`spiffe://<trust-domain>/<workload>`).
- Identity and access are different things. A SPIFFE SVID proves who a workload is — an OPA Rego policy decides what it is allowed to do. Both are required.
- SPIRE's attestation model is interesting — it verifies workload identity based on runtime attributes that are difficult to fake, like the exact container image digest or Kubernetes service account binding.

Relevance to Our Project

This paper is directly applicable to TrustMesh's CI/CD pipeline security module. Our plan is to use SPIFFE/SPIRE to issue ephemeral identities to build runners so they can access cloud resources without any stored secrets. The OPA integration described here also aligns with our policy engine design.

What We Plan to Apply

- Treat pipeline job runners as untrusted workloads and apply SPIRE node + workload attestation.
- Pair SPIFFE SVIDs with OPA Rego policies to authorize deployment operations dynamically.
- Remove all static cloud credentials from CI/CD variable stores.

Limitations / Gap

The paper is primarily a design-and-framework study. There are no empirical benchmarks showing how SPIRE attestation performs under heavy concurrent build loads. It also does not address the other side of the supply chain problem — artifact integrity, reproducible builds, or image signing — which is where the next few papers come in.

Paper 3 — Zero Trust in the Cloud: Implementing Zero Trust Architecture for Enhanced Cloud Security

Paper Details

- **Title:** Zero Trust in the Cloud: Implementing Zero Trust Architecture for Enhanced Cloud Security
- **Author:** Himanshu Sharma (Principal Software Engineer, Netskope Inc.)
- **Year / Publisher:** June 2022 | ESP Journal of Engineering & Technology Advancements (ESP-JETA), Vol. 2(2), pp. 78–91
- **DOI:** <https://doi.org/10.56472/25832646/JETA-V2I2P110>
- **Research Area:** Cloud Zero Trust Architecture, Micro-segmentation, Identity and Access Management

Problem Addressed

The author focuses on why the traditional 'castle-and-moat' security model breaks down in cloud environments. When infrastructure is distributed, when employees work remotely, and when workloads share cloud resources across regions, there is no clear boundary to defend. An attacker who gets inside — through a phishing attack, a compromised vendor, or a misconfigured public endpoint — can move freely because the internal network is trusted by default.

Approach / Methodology

Rather than building a prototype, this paper synthesizes NIST SP 800-207 guidelines into a practical cloud ZTA framework covering continuous identity verification, adaptive access control, micro-segmentation, and real-time telemetry. The author compares security posture metrics before and after ZTA adoption — measuring reduction in attack surface, unauthorized access attempts, and breach incidents across cloud infrastructure.

What We Understood

This paper is useful mainly as a broad conceptual reference. The key point is that Zero Trust is not a product you buy — it is a set of architectural decisions across identity, network, and monitoring. One thing the paper highlights that we found particularly relevant: once you try to manage workload identities across multiple cloud providers, you face a massive administrative burden. The author estimates tens of thousands of distinct user and workload identities in large enterprises, which shows why automation and standardization (like SPIFFE or OIDC federation) are not optional — they are necessary. The comparative metrics are encouraging: the paper shows measurable reduction in security incidents after ZTA adoption, which gives us some confidence that the approach is worth the implementation effort.

Relevance to Our Project

This paper validates the general direction of TrustMesh at a high level. It reinforces that continuous verification and micro-segmentation are worth implementing, and it highlights identity management complexity as a practical challenge we need to address with automation.

What We Plan to Apply

- Build continuous, context-aware verification into TrustMesh rather than one-time login-based trust.
- Use micro-segmentation to contain lateral movement in the event of a compromise.
- Automate identity lifecycle management to avoid manual key/credential tracking.

Limitations / Gap

The paper is deliberately high-level. There are no Kubernetes manifests, no GitOps workflows, and no specific implementation guidance. It also acknowledges that unified Zero Trust across multiple cloud providers is still an open challenge, which is exactly what our project is trying to address.

Paper 4 — Microsegmented Cloud Network Architecture Using Open-Source Tools for a Zero Trust Foundation

Paper Details

- **Title:** Microsegmented Cloud Network Architecture Using Open-Source Tools for a Zero Trust Foundation
- **Authors:** Sunil Arora and John Hastings (Dakota State University)
- **Year / Publisher:** 2024 | Dakota State University Research Publications (IEEE Format)
- **Research Area:** Cloud Network Micro-segmentation, Istio Service Mesh, Calico CNI, Open-Source Security

Problem Addressed

The problem here is about internal network security in multi-cloud Kubernetes environments. By default, Kubernetes networking is flat — any pod can talk to any other pod. Once an attacker compromises a single container, they can freely probe other services, extract secrets, or move toward databases and external APIs. The authors also specifically wanted to avoid proprietary vendor solutions since enterprise lock-in makes multi-cloud deployments inflexible.

Approach / Methodology

The researchers built a layered security architecture using only open-source tools. They paired Project Calico (for L3/L4 network policy — essentially packet-level firewall rules for pods) with Istio Service Mesh (for L7 application-layer identity and mTLS encryption). They set up a Kubernetes prototype with multiple namespaces (Foo, Bar, and Legacy) and tested traffic isolation. One important experiment they ran was switching Istio's PeerAuthentication mode from PERMISSIVE (allows plaintext) to STRICT (mTLS only), and observing that STRICT mode actively blocked plaintext connections while maintaining encrypted service communication.

What We Understood

- A single layer of network security is not enough. L3/L4 rules (Calico) restrict which pods can communicate at the packet level, but they cannot inspect application data. L7 Istio policies can verify cryptographic identity and encrypt traffic, but they do not stop low-level network attacks. Together, they create a real defense-in-depth.
- STRICT mTLS mode is important. PERMISSIVE mode might seem convenient during migration, but it creates a gap where services that have not yet onboarded to the mesh can still communicate in plaintext. In a Zero Trust design, this gap is unacceptable.
- Configuration drift is a real operational problem. Managing policies at pod level, namespace level, and global level across multiple teams creates inconsistencies over time. The paper does not solve this, but it explicitly calls it out as a major challenge.
- Certificate lifecycle management cannot be manual. With certificates constantly rotating across many microservices, automation is required — otherwise cert expiry causes outages.

Relevance to Our Project

This paper directly shows us what the service mesh layer of TrustMesh should look like. The Calico + Istio combination is essentially what we plan to implement for east-west traffic security in our Kubernetes clusters across AWS EKS and Azure AKS.

What We Plan to Apply

- Deploy dual-layer network controls: Calico for L3/L4 and Istio for L7 mTLS across namespaces.
- Enforce STRICT mTLS mode from the start — no PERMISSIVE mode in production clusters.
- Build automated policy validation into our GitOps pipeline to catch configuration drift.

Limitations / Gap

The prototype used simple mock HTTP services (httpbin) and was evaluated qualitatively. There are no performance numbers for CPU overhead, memory usage, or latency under realistic load — that data comes from Paper 8. The paper also does not address how CI/CD pipelines or IAM federation interact with the service mesh.

Paper 5 — Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM

Paper Details

- **Title:** Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM
- **Authors:** Jasmine Latendresse, Suhaib Mujahid, Diego Elias Costa, and Emad Shihab
- **Year / Publisher:** October 2022 | Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)
- **DOI:** <https://doi.org/10.1145/3551349.3556896>
- **Research Area:** Software Supply Chain Security, Software Composition Analysis (SCA), Dependency Reachability

Problem Addressed

This paper tackles alert fatigue in software composition analysis. Modern projects pull in dozens or hundreds of npm packages, and security scanners like Snyk or npm audit report vulnerabilities for every installed package — whether or not that package actually makes it into the final production bundle. The result is that developers spend time fixing vulnerabilities in packages that were never included in the deployed application in the first place.

Approach / Methodology

The authors studied 100 open-source JavaScript/NPM projects that use module bundlers (Webpack and Rollup) with tree-shaking. Tree-shaking removes unused code from production bundles. By analyzing build outputs, they could determine which installed dependencies actually ended up in the production artifact versus which were development-only tools that never shipped to users. They also benchmarked four widely-used SCA tools — npm audit, Snyk, Dependabot, and OWASP Dependency-Check — to see how many of their alerts pointed to packages that were not actually in production.

What We Understood

The findings here are quite striking. 51% of the projects they studied had zero production dependencies in the final bundle — every dependency was either a dev tool or got tree-shaken out. The remaining projects had a median of only 5 production-bundled packages. Yet SCA tools were flagging vulnerabilities across the entire install tree, creating enormous false-positive backlogs. The root cause is that tools like npm audit work against package.json and node_modules, not against the actual production bundle. None of the four tested tools could check bundle source maps or determine whether a vulnerable code path is actually reachable in the deployed artifact. For TrustMesh, this is directly applicable to our CI/CD pipeline — blindly failing builds on every SCA alert would be counterproductive and would lead developers to just ignore or suppress alerts entirely.

Relevance to Our Project

This paper changes how we think about dependency security scanning in TrustMesh's pipeline. Instead of blocking builds on every vulnerable package, we need to differentiate between packages that are in the production artifact and those that are development-only. This is what makes security gates actually useful rather than noisy.

What We Plan to Apply

- Distinguish between production-bundled and development-only dependencies in CI/CD security scans.
- Prioritize vulnerability alerts based on whether the affected code is reachable in the production artifact.
- Suppress non-production package alerts in deployment gates to reduce developer friction.

Limitations / Gap

The study is specific to the JavaScript/NPM ecosystem with tree-shaking bundlers. It does not cover compiled languages like Go or Rust, container base image packages, or OS-level dependencies — all of which are relevant in Kubernetes microservice deployments. Extending this reachability concept to those ecosystems is still an open problem.

Paper 6 — Reproducible Builds: Increasing the Integrity of Software Supply Chains

Paper Details

- **Title:** Reproducible Builds: Increasing the Integrity of Software Supply Chains
- **Authors:** Chris Lamb and Stefano Zacchiroli (Reproducible Builds Project)
- **Year / Publisher:** 2021/2022 | IEEE Software, Vol. 39, No. 2, pp. 62–70, IEEE Computer Society
- **DOI:** <https://doi.org/10.1109/MS.2021.3073024>
- **Research Area:** Software Supply Chain Integrity, Deterministic Compilation, Binary Verification

Problem Addressed

People typically review source code to verify software integrity, but end users run compiled binaries, not source code. There is nothing preventing a malicious build server from quietly modifying the output binary during compilation, even if the source code is clean. The SolarWinds attack is an example of exactly this — malicious code was injected during the build process, not in the source repository. If builds are non-deterministic (meaning the same source produces different binaries each time), there is no way for an independent party to verify that a published binary actually corresponds to its source.

Approach / Methodology

The paper describes the Reproducible Builds initiative and its tools for achieving deterministic compilation. The authors analyzed the root causes of build non-determinism in real-world software: embedded compile timestamps (like `__DATE__` and `__TIME__` in C), non-deterministic file ordering in archives, locale and timezone differences in build environments, and parallel build race conditions. They detail practical fixes used in the Debian Linux project: `SOURCE_DATE_EPOCH` (a standardized timestamp environment variable), `.buildinfo` manifests that record the build environment, `disorderfs` (for testing filesystem ordering sensitivity), and `diffoscope` (a binary diffing tool for diagnosing reproducibility failures). The result: over 95% reproducibility across 30,000+ Debian packages.

What We Understood

- Source code trust and binary trust are different things. Even if every line of source is audited, the build step is a separate trust boundary. A compromised build server can insert backdoors invisibly.
- Non-determinism in builds is surprisingly widespread. Most programmers do not think about it, but things like the system clock being embedded into a binary, or a hash map's iteration order being different each run, are enough to prevent reproducibility.
- Signing must be decoupled from building. If the same entity that builds the binary also signs it, there is no independent verification. The Reproducible Builds model allows independent parties to rebuild from source and compare the hash — if it matches, the binary is verified.
- The 95% reproducibility result from Debian is genuinely impressive given the diversity of packages, but it also means 5% are still non-deterministic — which shows this is hard to achieve completely.

Relevance to Our Project

This paper shapes how we approach container image signing in TrustMesh. Signing an image with Cosign/Sigstore is only meaningful if the build is deterministic — otherwise you cannot verify the image was built from clean source. It also highlights that we need to generate build provenance manifests (recording what went into the build) before signing.

What We Plan to Apply

- Pin base images and compiler/toolchain versions in Dockerfiles to reduce build variation.
- Use `SOURCE_DATE_EPOCH` normalization in container build processes where feasible.
- Generate and store cryptographic build provenance records before image signing (Cosign/Sigstore).

Limitations / Gap

The paper focuses on OS package distributions (specifically Debian) and C/C++ toolchains. Container orchestration, Kubernetes admission control, and runtime identity are out of scope. Applying these principles to Docker container builds has some differences that the paper does not specifically address.

Paper 7 — Taxonomy of Attacks on Open-Source Software Supply Chains

Paper Details

- **Title:** Taxonomy of Attacks on Open-Source Software Supply Chains
- **Authors:** Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais (SAP Security Research)
- **Year / Publisher:** 2023 | IEEE Transactions on Software Engineering (IEEE TSE)
- **DOI:** <https://doi.org/10.5281/zenodo.6395965>
- **Research Area:** Software Supply Chain Security, Attack Taxonomy, Threat Modelling, Pipeline Safeguards

Problem Addressed

Most development teams have some awareness of application-level vulnerabilities (like the OWASP Top 10), but supply chain attacks are far less understood. Attacks can happen at the source code stage, during the build, in the package registry, or at consumption time — and different attack vectors require different defences. Without a structured taxonomy, organizations do not know where to invest in protection or what the highest-risk attack paths are.

Approach / Methodology

The authors conducted a systematic literature review of 1,025 papers, security advisories, and MITRE ATT&CK entries. From this they built a formal attack tree taxonomy with 107 unique attack vectors mapped to 94 real-world supply chain incidents across four stages: Source Code, Build System, Package Distribution, and Downstream Consumption. They also identified 33 protective safeguards and evaluated their effectiveness, implementation cost, and developer awareness through surveys with 17 domain experts and 134 practitioners.

What We Understood

What stands out from this paper is the sheer scale of the attack surface. 107 distinct attack vectors means there is no single fix — supply chain security requires multiple layered controls. The most important insight for our project is the build system stage: cross-build injection (where a compromised earlier build poisons a later one by sharing cached state), build runner takeover, and malicious pipeline script injection are all real attack patterns that have occurred in practice. The survey data reveals an important gap: practitioners are well aware of code-level vulnerabilities but have very low awareness of build and registry attack vectors. This confirms that supply chain security education is itself a problem.

The safeguards rated as most cost-effective in the survey were: branch protection rules, removing unused dependencies, dependency version pinning, MFA for package registry accounts, and ephemeral build runners — all of which are things we can implement directly in TrustMesh's pipeline.

Relevance to Our Project

This paper provides the threat model for TrustMesh's pipeline security module. Rather than guessing what to protect, we now have a structured understanding of the attack surface. It confirms that our plan to use ephemeral runners, pin dependencies, sign artifacts, and verify SBOMs covers the highest-risk attack categories.

What We Plan to Apply

- Use ephemeral, isolated build runners for every pipeline job to prevent cross-build contamination.
- Enforce cryptographic dependency pinning (exact hashes, not just version numbers) in all manifests.
- Generate and verify SBOMs at build time and enforce SBOM-based admission in Kubernetes.

Limitations / Gap

The paper is a taxonomy and survey — it does not provide an automated platform or tool. It also focuses on open-source package registries (npm, PyPI, Maven) and does not address cloud-native workload identity or runtime Kubernetes security, which we must address separately.

Paper 8 — Performance Comparison of Service Mesh Frameworks: the mTLS Test Case

Paper Details

- **Title:** Technical Report: Performance Comparison of Service Mesh Frameworks: the mTLS Test Case
- **Authors:** Anat Bremler-Barr, Ofek Lavi, Yaniv Naor, Sanjeev Rampal, and Jhonatan Tavori (Tel Aviv University, Reichman University & Red Hat)
- **Year / Publisher:** 2024 | Technical Report (GitHub: [flikhamud45/sm-performance](https://github.com/flikhamud45/sm-performance))
- **Research Area:** Service Mesh Performance, mTLS Overhead, Sidecar vs. Sidecarless Architecture

Problem Addressed

Service meshes are widely recommended for implementing Zero Trust in Kubernetes — they handle automatic mTLS, traffic policy, and observability without changing application code. But implementing a service mesh is not free. Proxy sidecars consume memory, add CPU overhead, and introduce network hops that increase latency. The question this paper answers is: by how much, and does the choice of service mesh matter?

Approach / Methodology

The researchers ran controlled benchmarks on a Kubernetes cluster testing four service meshes: Istio with traditional Envoy sidecars, Istio Ambient (sidecarless mode with per-node Ztunnels), Linkerd (sidecar using a lightweight Rust proxy), and Cilium (eBPF-based, sidecarless). They measured P50 and P99 latency, CPU usage, and memory consumption at request rates from 200 to 12,800 RPS, for both intra-node (same host) and inter-node traffic.

What We Understood

The most counterintuitive result from this paper is that mTLS cryptographic encryption itself adds almost no latency. The overhead people blame on 'mTLS' is actually caused by proxy processing, inter-process communication, and extra network hops — not the TLS handshake itself. This distinction is important because it means the solution to the performance problem is not to skip mTLS, but to choose a lighter proxy architecture.

The concrete numbers are: Istio with Envoy sidecars added +166% latency at 3,200 RPS and consumed 355 MiB additional memory per client pod. Istio Ambient (sidecarless) added only +8% latency at the same load with dramatically less memory usage. Linkerd's Rust micro-proxy showed excellent CPU efficiency. Cilium performed best for inter-node CPU but had an important gap: its intra-node encryption was disabled by default, which is a security concern in shared-node environments.

Relevance to Our Project

This paper is very practically useful for TrustMesh. It tells us that if we use traditional Istio sidecars, we should expect significant performance degradation under load. We need to seriously consider Istio Ambient or Linkerd if we want mTLS without crippling throughput. This will be a key design decision when we implement the service mesh component.

What We Plan to Apply

- Evaluate Istio Ambient (sidecarless) as the primary service mesh option for TrustMesh to minimize latency impact.
- Benchmark our own deployment to verify performance under realistic workload patterns.
- Not disable intra-node mTLS — if using Cilium, ensure encryption is explicitly enabled for pod-to-pod traffic on the same node.

Limitations / Gap

The benchmark was done within a single Kubernetes cluster. Multi-cluster federation across AWS EKS and Azure AKS, cross-cloud identity validation, and OPA policy evaluation are not included in the measurements. Real-world multi-cloud performance may differ from these results.

Paper 9 — Zero Trust Security Model Implementation in Microservices Architectures Using Identity Federation

Paper Details

- **Title:** Zero Trust Security Model Implementation in Microservices Architectures Using Identity Federation
- **Authors:** Rethish Nair Rajendran, Sathish Krishna Anumula, Dileep Kumar Rai, and Sachin Agrawal
- **Year / Publisher:** 2025 | International Conference on Advancement in Science & Technology / IEEE Proceedings (DOI: 10.1109/ASSIC64892.2025.11158626)
- **Research Area:** Microservices Zero Trust, Identity Federation, SPIFFE/SPIRE, OPA Policy-as-Code

Problem Addressed

In a real microservice application, two things need to communicate: human users and the backend services they interact with. The problem is that these identities are usually handled separately. Users might authenticate with a login system (OAuth/OIDC), but backend services authenticating with each other often rely on static tokens or trusted network segments. Rajendran et al. address how to build a unified identity federation architecture that handles both human and non-human workload identity under a single Zero Trust model.

Approach / Methodology

The authors implemented an end-to-end Zero Trust framework following NIST SP 800-207 and SP 800-204. The stack they used is: Keycloak for user authentication (OIDC/OAuth 2.0 SSO), SPIFFE/SPIRE for non-human workload attestation, Istio for inter-service mTLS and JWT token enforcement at the proxy layer, and OPA with Rego policies managed through a GitOps pipeline for authorization decisions. They built a prototype with a React frontend, Node.js microservices, and MongoDB, and evaluated it through 15,000 authorization policy evaluations.

What We Understood

This paper is the closest to what TrustMesh is trying to build — it is essentially a student-scale implementation of the same architecture stack we have planned. The numbers reported are quite strong: OPA Rego policy evaluation achieved 99.9% accuracy across the 15,000 test evaluations, unauthorized access attempts dropped by over 90%, and the estimated breach probability reduction was 81.8%. The latency overhead from Istio and OPA combined was described as marginal and acceptable for cloud microservices — though specific numbers for high-throughput scenarios are not provided.

The GitOps policy management approach is something we found very useful. Storing Rego policies in Git, testing them in CI, and deploying them automatically via ArgoCD or Flux means policies go through the same review process as application code, which is exactly what Zero Trust governance requires.

Relevance to Our Project

This paper is probably the most directly relevant to our work. It demonstrates that the Keycloak + SPIFFE/SPIRE + Istio + OPA combination works in practice and delivers measurable security improvements. We can use the architecture and testing methodology as a reference design for TrustMesh.

What We Plan to Apply

- Federate human identity (OIDC/Keycloak) with workload identity (SPIFFE SVIDs) in inter-service calls.
- Manage OPA Rego policies through GitOps pipelines with automated testing before deployment.
- Use Istio to enforce mTLS and intercept/validate JWT tokens at the service mesh proxy layer.

Limitations / Gap

The prototype ran on a single Kubernetes cluster. Cross-cloud federation between AWS EKS and Azure AKS was not demonstrated. CI/CD supply chain security and container image signing are also not covered, which is the gap our TrustMesh project aims to fill by combining this runtime architecture with the supply chain controls from Papers 5, 6, and 7.

Paper 10 — Survivable Zero Trust for Cloud Computing Environments

Paper Details

- **Title:** Survivable zero trust for cloud computing environments
- **Authors:** Luca Ferretti, Federico Magnanini, Mauro Andreolini, and Michele Colajanni
- **Year / Publisher:** November 2021 | Computers & Security, Elsevier, Vol. 110, Article 102419
- **DOI:** <https://doi.org/10.1016/j.cose.2021.102419>
- **Research Area:** Zero Trust Control Plane Survivability, Byzantine Fault Tolerance, Threshold Cryptography

Problem Addressed

This paper raises a critical architectural question: what happens if the Zero Trust control plane itself is compromised? Most Zero Trust frameworks (NIST SP 800-207, Google BeyondCorp) assume that the Policy Decision Point (PDP) and Identity Provider (IdP) are trustworthy. But if an attacker compromises the PDP server or steals the IdP's private signing key, the entire ZTA collapses — the attacker can issue valid tokens or approve any access request. This is a single point of failure that most ZTA designs do not address.

Approach / Methodology

The authors designed what they call a 'Survivable Zero Trust Architecture' (SZTA) that applies Zero Trust principles to the control plane itself. They used Byzantine Fault Tolerant (BFT) state machine replication for policy stores and device inventories, so that no single server can unilaterally alter policy decisions. For the IdP signing key, they applied the PROTECT threshold cryptography protocol — splitting the private key across n servers so that at least $f+1$ servers must cooperate to sign a token (meaning an attacker would need to compromise multiple servers simultaneously). They also integrated Single Packet Authorization (SPA) to hide control plane infrastructure from network scanners. The feasibility was tested by measuring user authentication and request authorization latency.

What We Understood

The most important insight from this paper is something we had not thought about before reading it: Zero Trust verification applies to the control plane too, not just to user and workload traffic. This is a logical extension of 'never trust, always verify' — you cannot exempt your own policy engine from that principle. The threshold cryptography approach is elegant: even if an attacker compromises one IdP server, they cannot forge tokens because the signing key does not exist in one place.

The trade-off, though, is significant. Authentication latency was under 2 seconds, but authorization latency was 100–300 ms per request. For microservices handling thousands of requests per second, 300 ms per policy decision would be completely unacceptable. This is the core tension the paper does not fully resolve — how do you apply Byzantine fault tolerance to a latency-sensitive authorization path?

Relevance to Our Project

This paper gives us a useful warning for TrustMesh's policy engine and identity issuer design. We should not build TrustMesh with a single OPA instance or a single SPIRE server as a potential point of compromise. High availability, redundancy, and policy integrity verification are things we need to plan for, even if we do not implement full BFT in our student-level prototype.

What We Plan to Apply

- Deploy OPA and SPIRE in HA configurations with multiple replicas rather than single-instance setups.
- Implement continuous policy integrity checks to detect unauthorized tampering of Rego rules in the policy store.
- Accept the latency limitation of full BFT as out of scope for TrustMesh, but document it as a future production consideration.

Limitations / Gap

The 100–300 ms authorization latency makes BFT control planes impractical for high-throughput microservices in their current form. The paper also does not address Kubernetes container orchestration, CI/CD supply chain defense, or multi-cloud identity federation — which are the central concerns of our project.

Overall Understanding from the 10 Research Papers

After going through all 10 papers, the clearest takeaway is that Zero Trust is not a product, a single tool, or even a configuration — it is an architectural discipline that spans identity, networking, authorization, supply chain, and monitoring. No single paper covers all of these. Each paper addressed one or two aspects in depth, which is exactly why we needed all ten to build a complete picture.

A. Identity is the New Security Boundary

Papers 1, 2, 3, and 9 consistently make the same point: network location can no longer be trusted as a security boundary. Static credentials — IAM keys stored in Kubernetes secrets, shared runner tokens, hardcoded service passwords — are fundamentally incompatible with Zero Trust. The move to ephemeral, cryptographically bound tokens (OIDC JWTs, SPIFFE SVIDs) is not optional — it is the foundation. What is interesting is how Papers 1 and 9 complement each other: Paper 1 handles machine-to-cloud authentication, while Paper 9 handles the more complete picture of unifying human SSO with workload identity into a single federation model.

B. Kubernetes Security Requires Both Network and Application Layer Controls

Papers 3, 4, and 8 together show that Kubernetes environments need security at multiple network layers. Paper 4 demonstrates the Calico + Istio pairing for dual-layer isolation. Paper 8 then gives us the performance reality check — not all service mesh configurations are equally efficient. The surprising finding from Paper 8 is that sidecarless architectures are not just equally secure, they are significantly faster. This changes the conventional assumption that the most secure option (full Envoy sidecars) is the default choice.

C. Policy-as-Code is What Turns Identity Into Access Control

Papers 2, 9, and to some extent 10 all make the point that identity alone does not grant access. OPA with Rego policies is what converts a verified SPIFFE ID or OIDC token into an allow or deny decision. Managing these policies through GitOps (Git-controlled, reviewed, tested, and automatically deployed) is highlighted in Paper 9 as what makes the system auditable and scalable.

D. Supply Chain Security is a Separate (and Often Neglected) Problem

Papers 5, 6, and 7 deal with an aspect that the runtime Zero Trust papers largely ignore: what if the container image that gets deployed was tampered with before it even reached the cluster? Paper 7's taxonomy of 107 supply chain attack vectors shows just how many ways this can happen. Paper 6 addresses how to verify build integrity through reproducible builds and provenance records. Paper 5 shows that blindly scanning all dependencies without differentiating production from dev creates alert fatigue that defeats the purpose of scanning. Together, these three papers tell us: Zero Trust must start at the CI/CD pipeline, not just at Kubernetes admission.

E. The Control Plane Itself Is a Vulnerability

Paper 10 is the one that most changed how we think about TrustMesh's architecture. Most Zero Trust designs (including the ones in Papers 3, 4, and 9) implicitly trust the policy engine and identity issuer. Paper 10 argues this is a mistake. While we will not implement Byzantine Fault Tolerance in TrustMesh (the latency cost is too high for a student project), we will design for high availability and policy integrity monitoring, informed by this paper's argument.

Common Research Gaps Identified

Looking across all 10 papers, we identified four recurring gaps where the literature does not connect the dots between the areas it covers:

1. No Link Between Build Provenance and Runtime Workload Identity

Papers 5, 6, and 7 focus on build integrity and dependency security before deployment. Papers 1, 2, and 9 focus on identity and policy after deployment. There is no paper that connects these two phases — specifically, no mechanism that ties a verified build attestation (signed SBOM, reproducible build digest) directly to the SPIFFE SVID issuance decision at runtime. In current systems, once a container passes admission, runtime identity is granted based on pod labels or service accounts without verifying the container image's build history. This is the gap TrustMesh specifically aims to explore.

2. Multi-Cloud Authorization Not Fully Addressed

Paper 1 demonstrates cross-cloud authentication federation across AWS, GCP, and Azure. But authentication is only the first step. Attribute-Based Access Control (ABAC) that translates federated identity claims into context-aware authorization policies across AWS EKS and Azure AKS simultaneously is not addressed in any of the papers. Papers that cover OPA (2, 9) do so within single-cluster environments.

3. Policy Drift Across Distributed Teams and Clusters

Papers 3 and 4 explicitly acknowledge configuration drift as a major operational problem. Policies managed by multiple teams across multiple Kubernetes clusters in multiple clouds tend to diverge over time. None of the reviewed papers present an automated GitOps-based reconciliation engine that continuously detects and remediates policy drift across hybrid AWS/Azure environments. This is a gap TrustMesh can partially address through GitOps-managed policy distribution.

4. No Balanced Architecture for mTLS Performance vs. Strong Identity

Paper 8 shows that traditional Envoy sidecars have severe latency overhead (+166%), while Paper 10 shows that Byzantine-resilient control planes have authorization latency of 100–300 ms. There is no paper that proposes a balanced architecture delivering strong cryptographic workload identity with mTLS while maintaining acceptable throughput for production microservices. This is an engineering trade-off we will need to navigate in TrustMesh.

How TrustMesh Addresses These Gaps

TrustMesh is not a production system — it is a final-year project prototype exploring how these gaps can be bridged. Our planned approach is to: (1) connect CI/CD build attestation with runtime identity issuance through signed provenance records checked at admission; (2) implement SPIFFE/OIDC identity coupled with OPA Rego policies across both AWS EKS and Azure AKS; (3) use GitOps (ArgoCD or Flux) to maintain consistent policy distribution across clusters; and (4) evaluate Istio Ambient's sidecarless architecture as a way to reduce mTLS latency compared to traditional sidecars.

Mapping the Research to TrustMesh Components

The table below maps each research paper's primary contribution to the corresponding component of TrustMesh and what we plan to derive from it:

1	Deochake et al. — Multi-cloud OIDC workload identity federation	Identity Engine: ephemeral token design for AWS EKS & Azure AKS
2	Avirneni — SPIFFE/SPIRE identity for CI/CD pipeline runners	Pipeline Security: secretless build runner authentication using SVIDs
3	Sharma — ZTA overview, NIST SP 800-207 in cloud environments	Architecture baseline: continuous verification & micro-segmentation principles
4	Arora & Hastings — Calico + Istio dual-layer micro-segmentation	Service Trust: L3/L4 Calico policies + L7 Istio mTLS configuration
5	Latendresse et al. — Production vs. dev dependency reachability	Pipeline Security: context-aware SCA gating, suppressing dev-only alerts
6	Lamb & Zacchioli — Reproducible builds and provenance records	Pipeline Security: deterministic image builds, Cosign image signing
7	Ladisa et al. — Taxonomy of 107 supply chain attack vectors	Threat model: ephemeral runners, dependency pinning, SBOM enforcement
8	Bremner-Barr et al. — Service mesh mTLS performance benchmarks	Service Trust: sidecarless (Istio Ambient) selection to reduce latency
9	Rajendran et al. — OIDC + SPIFFE + Istio + OPA integration prototype	Full stack reference: GitOps OPA policies, Istio JWT enforcement
10	Ferretti et al. — BFT control plane survivability and policy integrity	Verification Engine: HA policy store design, policy tamper detection

What We Take Forward into ZeroTrustCloud

Based on our reading of the 10 papers, we have identified the following key technical decisions that will guide the design and implementation of TrustMesh. These are not generic best practices — each one is grounded in a specific finding from the reviewed literature.

- Cryptographic workload identity must replace network-based trust entirely. IP addresses and network segments are not security boundaries in Kubernetes. Every workload will have a cryptographic identity (SPIFFE SVID or OIDC JWT) — no exceptions. (Papers 1, 2, 9)
- Static credentials are banned from TrustMesh. All IAM keys, service account private keys, and stored runner tokens will be replaced with ephemeral, audience-bound tokens with a maximum 60-minute lifetime. (Papers 1, 2)
- SPIFFE/SPIRE will be used for non-human workload identity attestation — both for microservices and CI/CD pipeline runners. Runtime attestation using container selectors ensures identities are tied to what is actually running. (Papers 2, 9)
- OPA with Rego policies managed via GitOps will handle all authorization decisions. Policy-as-Code, version-controlled and automatically deployed, ensures that authorization rules are auditable and consistent. (Papers 2, 9)
- STRICT mTLS will be enforced across all inter-service communication. There will be no PERMISSIVE mode in production clusters. Istio Ambient (sidecarless) will be evaluated first to keep latency overhead acceptable. (Papers 4, 8)
- Pipeline runners will be treated as untrusted workloads. Every build job will run in an ephemeral environment with the minimum permissions needed, and will authenticate via SPIRE rather than stored secrets. (Papers 2, 7)
- Dependency scanning in CI/CD will differentiate production from dev dependencies. We will configure SCA tools to prioritize alerts on production-bundled packages and suppress dev-only alerts. (Paper 5)
- Container images will be built deterministically and signed before admission. We will use Cosign/Sigstore for image signing and generate build provenance records at build time. (Papers 6, 7)
- TrustMesh's policy store and identity issuer will not be single-instance deployments. Based on Paper 10's warning about control plane single points of failure, we will run OPA and SPIRE in HA configurations with multiple replicas.
- Multi-cloud consistency across AWS EKS and Azure AKS will be maintained through GitOps-managed policy distribution. Drift detection will be automated as part of the verification layer. (Papers 1, 3, 4)

References

The following 10 research papers form the primary source material for this literature review. All details are verified against the original documents.

- [1] Deochake, S., Murphy, R., & Gearheart, J. (2024). A Multi-Cloud Framework for Zero-Trust Workload Authentication. SentinelOne Inc. Technical Research Publications.
- [2] Avirneni, S. T. (2025). Establishing Workload Identity for Zero Trust CI/CD: From Secrets to SPIFFE-Based Authentication. Independent Cloud Security & Platform Architecture Technical Report, April 2025.
- [3] Sharma, H. (2022). Zero Trust in the Cloud: Implementing Zero Trust Architecture for Enhanced Cloud Security. *ESP Journal of Engineering & Technology Advancements (ESP-JETA)*, 2(2), 78–91. <https://doi.org/10.56472/25832646/JETA-V2I2P110>
- [4] Arora, S., & Hastings, J. (2024). Microsegmented Cloud Network Architecture Using Open-Source Tools for a Zero Trust Foundation. Dakota State University Research Publications, IEEE Format.
- [5] Latendresse, J., Mujahid, S., Costa, D. E., & Shihab, E. (2022). Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. <https://doi.org/10.1145/3551349.3556896>
- [6] Lamb, C., & Zacchiroli, S. (2022). Reproducible Builds: Increasing the Integrity of Software Supply Chains. *IEEE Software*, 39(2), 62–70. <https://doi.org/10.1109/MS.2021.3073024>
- [7] Ladisa, P., Plate, H., Martinez, M., & Barais, O. (2023). Taxonomy of Attacks on Open-Source Software Supply Chains. *IEEE Transactions on Software Engineering*. <https://doi.org/10.5281/zenodo.6395965>
- [8] Bremner-Barr, A., Lavi, O., Naor, Y., Rampal, S., & Tavori, J. (2024). Technical Report: Performance Comparison of Service Mesh Frameworks: the MTLS Test Case. Tel Aviv University, Reichman University & Red Hat. <https://github.com/flikhamud45/sm-performance>
- [9] Rajendran, R. N., Anumula, S. K., Rai, D. K., & Agrawal, S. (2025). Zero Trust Security Model Implementation in Microservices Architectures Using Identity Federation. *IEEE ASSIC 2025*. <https://doi.org/10.1109/ASSIC64892.2025.11158626>
- [10] Ferretti, L., Magnanini, F., Andreolini, M., & Colajanni, M. (2021). Survivable zero trust for cloud computing environments. *Computers & Security*, 110, Article 102419. Elsevier. <https://doi.org/10.1016/j.cose.2021.102419>